# VELLORE INSTITUTE OF TECHNOLOGY

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

## BCSE302L – DATABASE SYSTEMS

Course Project

## REVIEW II

Database Implementation

# Disaster Management and Emergency Response System (DMERS)

*Implementation of the Incident-Response Database*


| Submitted by | Register Number |
|---|---|
| Vighnesh Das | 24BYB0153 |
| Soumyadeep Chowdhury | 24BYB0175 |


| | |
|---|---|
| **Course** | BCSE302L Database Systems (Course Project) |
| **Semester** | Fall Semester 2026–27 |
| **Review stage** | Review II – Database Implementation |
| **Database platform** | PostgreSQL 16 with PL/pgSQL, executed through psql |
| **Date** | October 2026 |

## CONTENTS


| No. | Section | Review II component (marks) | Implemented (course minimum) | Page |
|---|---|---|---|---|
| 1 | Implementation Overview | – | Scripts 00–07, run in numeric order | 2 |
| 2 | DDL and Constraints | DDL & constraints (2) | 8 tables (6–8); 8 PK, 11 FK, 5 UQ, 24 CHECK, 14 DEFAULT | 2 |
| 3 | DML and Data Population | DML & data population (1) | 12–61 rows per table (10); INSERT/UPDATE/DELETE | 4 |
| 4 | SQL Queries | SQL queries (2) | 15 queries (15) | 5 |
| 5 | Procedures and Functions | Procedures & functions (2) | 2 procedures, 3 functions (2, 2) | 7 |
| 6 | Cursor and Triggers | Cursor & triggers (2) | 1 cursor (1), 5 triggers (2) | 8 |
| 6.3 | Execution and Transaction Behaviour | Viva / technical understanding (1) | Execution flow of six operations | 10 |
| 7 | Conclusion, Next Steps and References | – | – | 10 |


# 1. IMPLEMENTATION OVERVIEW

Review II implements the Review I design in PostgreSQL 16: the eight relations, keys and domains of Review I (Tables 12–14), time-ordered demonstration data, fifteen queries, and the PL/pgSQL functions, procedures, cursor and triggers that Review I assigned to the business rules. The scripts *00_reset* to *07_constraint_tests* run in psql in numeric order, or together through *run_all.sql*, in a separate course database, *dmers_course*, so the application database *dmers* of the Flask backend is never altered (*00_reset* refuses to run there). Listings are copied from the scripts; every output comes from running them on a newly created database, with psql's script-location prefix on notices and errors omitted.

# 2. DDL AND CONSTRAINTS

## 2.1 Table Definitions

Constraints are named by kind (pk_, fk_, uq_, ck_), so a rejected statement names the rule it broke. Strong relations use identity surrogate keys; foreign-key actions follow Review I, Table 13 (RESTRICT, CASCADE to the weak relation *incident_updates*, SET NULL for the optional unit lead). *ck_incidents_zone_when_verified* ties *status* to *zone_id* (FR-05), and the partial unique index *uq_dispatches_one_active_per_unit* allows only one active dispatch per unit (BR-05).

**Listing 1. Table definitions (01_schema.sql without its header)**

```
CREATE TABLE users (
  id            INTEGER GENERATED ALWAYS AS IDENTITY
                CONSTRAINT pk_users PRIMARY KEY,
  full_name     VARCHAR(100) NOT NULL,
  account_type  VARCHAR(15) NOT NULL DEFAULT 'CITIZEN'
    CONSTRAINT ck_users_account_type CHECK (account_type
      IN ('CITIZEN', 'COORDINATOR', 'UNIT_LEAD',
          'ADMINISTRATOR')),
  email         VARCHAR(120) NOT NULL
    CONSTRAINT uq_users_email UNIQUE
    CONSTRAINT ck_users_email_format CHECK (email ~*
      '^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$'),
  password_hash TEXT NOT NULL,        -- hash only (BR-01)
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
CREATE TABLE zones (
  id            INTEGER GENERATED ALWAYS AS IDENTITY
                CONSTRAINT pk_zones PRIMARY KEY,
  zone_name     VARCHAR(60) NOT NULL
                CONSTRAINT uq_zones_name UNIQUE,
  district      VARCHAR(60) NOT NULL,
  state         VARCHAR(60) NOT NULL DEFAULT 'Tamil Nadu',
  risk_level    VARCHAR(6) NOT NULL
    CONSTRAINT ck_zones_risk_level
      CHECK (risk_level IN ('LOW', 'MEDIUM', 'HIGH'))
);
```

```
CREATE TABLE incidents (
  id            INTEGER GENERATED ALWAYS AS IDENTITY
                CONSTRAINT pk_incidents PRIMARY KEY,
  title         VARCHAR(120) NOT NULL,
  description   TEXT,
  incident_type VARCHAR(30) NOT NULL,
  severity      VARCHAR(8) NOT NULL DEFAULT 'MODERATE'
    CONSTRAINT ck_incidents_severity CHECK (severity
      IN ('LOW', 'MODERATE', 'HIGH', 'CRITICAL')),
  status        VARCHAR(10) NOT NULL DEFAULT 'REPORTED'
    CONSTRAINT ck_incidents_status CHECK (status
      IN ('REPORTED', 'VERIFIED', 'RESPONDING',
          'RESOLVED', 'CLOSED', 'REJECTED')),
  location      VARCHAR(150) NOT NULL,
  zone_id       INTEGER           -- NULL until verified
    CONSTRAINT fk_incidents_zone
      REFERENCES zones (id) ON DELETE RESTRICT,
  reported_by   INTEGER NOT NULL
    CONSTRAINT fk_incidents_reporter
      REFERENCES users (id) ON DELETE RESTRICT,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT ck_incidents_zone_when_verified   -- FR-05
    CHECK (status IN ('REPORTED', 'REJECTED')
          OR zone_id IS NOT NULL),
  CONSTRAINT ck_incidents_timestamps
    CHECK (updated_at >= created_at)
);
```

```
CREATE TABLE response_units (
  id            INTEGER GENERATED ALWAYS AS IDENTITY
                CONSTRAINT pk_response_units PRIMARY KEY,
  unit_name     VARCHAR(60) NOT NULL
                CONSTRAINT uq_units_name UNIQUE,
  unit_type     VARCHAR(10) NOT NULL
    CONSTRAINT ck_units_type CHECK (unit_type IN
      ('FIRE', 'MEDICAL', 'POLICE', 'RESCUE', 'RELIEF')),
  contact_phone VARCHAR(10) NOT NULL
    CONSTRAINT ck_units_phone
      CHECK (contact_phone ~ '^[6-9][0-9]{9}$'),
  personnel_count SMALLINT NOT NULL
    CONSTRAINT ck_units_personnel
      CHECK (personnel_count > 0),
  availability_status VARCHAR(10) NOT NULL
    DEFAULT 'AVAILABLE'
    CONSTRAINT ck_units_availability
      CHECK (availability_status IN
        ('AVAILABLE', 'DEPLOYED', 'OFF_DUTY')),
  zone_id       INTEGER NOT NULL    -- total in BASED_IN
    CONSTRAINT fk_units_zone
      REFERENCES zones (id) ON DELETE RESTRICT,
  lead_user_id  INTEGER             -- optional 1:1 LEADS
    CONSTRAINT uq_units_lead UNIQUE
    CONSTRAINT fk_units_lead
```

```
      REFERENCES users (id) ON DELETE SET NULL
);

CREATE TABLE resources (
  id            INTEGER GENERATED ALWAYS AS IDENTITY
                CONSTRAINT pk_resources PRIMARY KEY,
  resource_name VARCHAR(60) NOT NULL
                CONSTRAINT uq_resources_name UNIQUE,
  category      VARCHAR(10) NOT NULL
    CONSTRAINT ck_resources_category CHECK (category IN
      ('FOOD', 'WATER', 'MEDICAL', 'SHELTER',
       'EQUIPMENT')),
  unit_of_measure VARCHAR(20) NOT NULL,
  quantity_available INTEGER NOT NULL DEFAULT 0
    CONSTRAINT ck_resources_quantity
      CHECK (quantity_available >= 0),
  reorder_level INTEGER NOT NULL DEFAULT 0
    CONSTRAINT ck_resources_reorder
      CHECK (reorder_level >= 0)
);

CREATE TABLE incident_updates (    -- weak INCIDENT_UPDATE
  incident_id   INTEGER NOT NULL
    CONSTRAINT fk_updates_incident
      REFERENCES incidents (id) ON DELETE CASCADE,
  update_no     INTEGER NOT NULL       -- partial key
    CONSTRAINT ck_updates_no CHECK (update_no >= 1),
  previous_status VARCHAR(10)
    CONSTRAINT ck_updates_previous_status
      CHECK (previous_status IN ('REPORTED', 'VERIFIED',
        'RESPONDING', 'RESOLVED', 'CLOSED', 'REJECTED')),
  new_status    VARCHAR(10) NOT NULL
    CONSTRAINT ck_updates_new_status
      CHECK (new_status IN ('REPORTED', 'VERIFIED',
        'RESPONDING', 'RESOLVED', 'CLOSED', 'REJECTED')),
  note          VARCHAR(200),
  logged_by     INTEGER          -- NULL: no signed-in user
    CONSTRAINT fk_updates_author
      REFERENCES users (id) ON DELETE RESTRICT,
  logged_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT pk_incident_updates
    PRIMARY KEY (incident_id, update_no),
  CONSTRAINT ck_updates_first_entry
    CHECK ((update_no = 1) = (previous_status IS NULL))
);

CREATE TABLE dispatches (  -- ternary DISPATCHED_TO, M:N:1
  incident_id   INTEGER NOT NULL
    CONSTRAINT fk_dispatches_incident
      REFERENCES incidents (id) ON DELETE RESTRICT,
  unit_id       INTEGER NOT NULL
    CONSTRAINT fk_dispatches_unit
      REFERENCES response_units (id) ON DELETE RESTRICT,
  dispatched_by INTEGER NOT NULL  -- authorising official
    CONSTRAINT fk_dispatches_coordinator
      REFERENCES users (id) ON DELETE RESTRICT,
  dispatch_status VARCHAR(10) NOT NULL DEFAULT 'ASSIGNED'
    CONSTRAINT ck_dispatches_status
      CHECK (dispatch_status IN ('ASSIGNED', 'EN_ROUTE',
        'ON_SCENE', 'COMPLETED', 'CANCELLED')),
  dispatched_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  arrived_at    TIMESTAMPTZ,
  cleared_at    TIMESTAMPTZ,
  CONSTRAINT pk_dispatches
    PRIMARY KEY (incident_id, unit_id),
  CONSTRAINT ck_dispatches_arrival               -- BR-06
    CHECK (arrived_at >= dispatched_at),
  CONSTRAINT ck_dispatches_clearance
    CHECK (cleared_at >=
           coalesce(arrived_at, dispatched_at)),
  CONSTRAINT ck_dispatches_arrived_on_scene
    CHECK (dispatch_status NOT IN ('ON_SCENE','COMPLETED')
           OR arrived_at IS NOT NULL),
  CONSTRAINT ck_dispatches_cleared_when_closed
    CHECK ((dispatch_status IN ('COMPLETED', 'CANCELLED'))
           = (cleared_at IS NOT NULL))
);

CREATE TABLE resource_allocations (       -- M:N ALLOCATED
  incident_id   INTEGER NOT NULL
    CONSTRAINT fk_allocations_incident
      REFERENCES incidents (id) ON DELETE RESTRICT,
  resource_id   INTEGER NOT NULL
    CONSTRAINT fk_allocations_resource
      REFERENCES resources (id) ON DELETE RESTRICT,
  quantity      INTEGER NOT NULL
    CONSTRAINT ck_allocations_quantity
      CHECK (quantity > 0),
  allocated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT pk_resource_allocations
    PRIMARY KEY (incident_id, resource_id)
);

-- BR-05 backstop: at most one active dispatch per unit
CREATE UNIQUE INDEX uq_dispatches_one_active_per_unit
  ON dispatches (unit_id)
  WHERE dispatch_status IN ('ASSIGNED', 'EN_ROUTE',
                            'ON_SCENE');

-- Indexes on foreign keys and the most frequent filter
CREATE INDEX ix_incidents_status_zone
  ON incidents (status, zone_id);
CREATE INDEX ix_incidents_reported_by
  ON incidents (reported_by);
CREATE INDEX ix_units_zone ON response_units (zone_id);
CREATE INDEX ix_dispatches_unit ON dispatches (unit_id);
CREATE INDEX ix_allocations_resource
  ON resource_allocations (resource_id);
```

## 2.2 Constraint Verification

*07_constraint_tests.sql* attempts fourteen changes that break the design. The constraint names and SQLSTATE codes in Table 1 were read from the errors returned to the client; every statement was rolled back.

**Table 1. Statements rejected by the database**

| No. | Attempted change | Constraint that rejected it | SQLSTATE |
|---|---|---|---|
| K1 | Duplicate e-mail address | `uq_users_email` | 23505 |
| K2 | Account type outside the domain | `ck_users_account_type` | 23514 |
| K3 | Malformed e-mail address | `ck_users_email_format` | 23514 |
| K4 | Severity outside the domain | `ck_incidents_severity` | 23514 |
| K5 | Verifying an incident without a zone | `ck_incidents_zone_when_verified` | 23514 |
| K6 | Report from a user that does not exist | `fk_incidents_reporter` | 23503 |
| K7 | Response unit with no personnel | `ck_units_personnel` | 23514 |
| K8 | Malformed contact number | `ck_units_phone` | 23514 |
| K9 | Negative stock | `ck_resources_quantity` | 23514 |
| K10 | Allocation of zero items | `ck_allocations_quantity` | 23514 |
| K11 | Arrival recorded before the dispatch time | `ck_dispatches_arrival` | 23514 |
| K12 | Second active dispatch for one unit (direct inserts) | `uq_dispatches_one_active_per_unit` | 23505 |
| K13 | Deleting a user who reported incidents (RESTRICT) | `fk_incidents_reporter` | 23503 |
| K14 | Deleting an incident with dispatch history (BR-09) | `fk_dispatches_incident` | 23503 |

*23505 unique_violation; 23514 check_violation; 23503 foreign_key_violation. K12 uses two direct INSERTs, which bypass the procedure that marks a unit DEPLOYED; the partial unique index still refuses the second active dispatch.*

# 3. DML AND DATA POPULATION

## 3.1 Data Population

*03_seed_data.sql* inserts the reference data with multi-row INSERTs, then replays the incidents of 1–12 October 2026 in time order (reports, verifications, dispatches, arrivals, allocations, closures), each with its own timestamp. The triggers already exist, so they write the timeline, move the stock and check every dispatch during the load. All people and contacts are fictitious.

**Table 2. Rows per relation after the load**

| Relation | Rows | What the rows represent |
|---|---|---|
| users | 16 | citizen 7, unit lead 5, coordinator 3, administrator 1; password hashes only |
| zones | 12 | Vellore, Ranipet and Tirupattur districts; risk high 3, low 3, medium 6; *state* from its DEFAULT |
| incidents | 14 | 1–12 October 2026; closed 4, responding 3, reported 2, resolved 2, verified 2, rejected 1 |
| response_units | 12 | fire 3, medical 3, police 2, relief 2, rescue 2; 5 with a lead; available 4, deployed 7, off duty 1 |
| resources | 12 | five categories; after the allocations 1 out of stock, 3 low, 8 adequate |
| incident_updates | 61 | written by triggers during the load: 45 status changes, 16 allocation notes |
| dispatches | 16 | 9 completed or cancelled, 7 active |
| resource_allocations | 15 | 6 incidents, 12 items; one cumulative allocation (DD-07) |

## 3.2 Insert, Update and Delete

The INSERT relies on the defaults for *status* and *severity*, and trigger T1 writes timeline entry 1; verification must assign a zone (*ck_incidents_zone_when_verified*) and T2 sets *updated_at*. The DELETE removes rejected report 8 and its timeline by cascade; incident 3, which has dispatch history, cannot be deleted (Table 1, K14).

**Listing 2. INSERT, UPDATE and DELETE with their outputs (04_dml_demo.sql)**

**INSERT – a citizen reports an incident (FR-03)**

```
INSERT INTO incidents (title, description, incident_type, location, reported_by)
VALUES ('Waterlogging under Katpadi railway underpass', 'Two-wheelers stalled in water',
        'FLOOD', 'Railway underpass, Katpadi', 16)
RETURNING id, status, severity, zone_id, to_char(created_at, 'DD-Mon HH24:MI') AS reported;
SELECT update_no, previous_status, new_status, note, logged_by
FROM incident_updates WHERE incident_id = 15;
```

```
 id |  status  | severity | zone_id |   reported
----+----------+----------+---------+--------------
 15 | REPORTED | MODERATE |         | 12-Oct 20:09

INSERT 0 1
 update_no | previous_status | new_status |       note        | logged_by
-----------+-----------------+------------+-------------------+-----------
         1 |                 | REPORTED   | Incident reported |        16
```

**UPDATE – verification (FR-05) and a unit's arrival (FR-08)**

```
SET dmers.actor_id = '3';           -- coordinator acting
UPDATE incidents
SET status = 'VERIFIED', severity = 'HIGH', zone_id = 2
WHERE id = 15
RETURNING id, status, severity, zone_id,
          to_char(updated_at, 'DD-Mon HH24:MI') AS updated;
UPDATE dispatches
SET dispatch_status = 'ON_SCENE', arrived_at = now()
WHERE incident_id = 7 AND unit_id = 11
RETURNING incident_id, unit_id, dispatch_status;
```

```
SET
 id |  status  | severity | zone_id |   updated
----+----------+----------+---------+--------------
 15 | VERIFIED | HIGH     |       2 | 12-Oct 20:09

UPDATE 1
 incident_id | unit_id | dispatch_status
-------------+---------+-----------------
           7 |      11 | ON_SCENE

UPDATE 1
```

**UPDATE – restocking (FR-09)**

```
UPDATE resources
SET quantity_available = quantity_available + 100
WHERE resource_name = 'Tarpaulin sheets';
SELECT resource_name, quantity_available,
       fn_stock_status(id) AS stock_status
FROM resources WHERE resource_name = 'Tarpaulin sheets';
```

```
UPDATE 1
  resource_name   | quantity_available | stock_status
------------------+--------------------+--------------
 Tarpaulin sheets |                150 | ADEQUATE
```

**DELETE – the rejected duplicate report**

```
SELECT count(*) AS timeline_rows FROM incident_updates WHERE incident_id = 8;
DELETE FROM incidents WHERE id = 8 AND status = 'REJECTED';
SELECT count(*) AS timeline_rows FROM incident_updates WHERE incident_id = 8;
```

```
 timeline_rows
---------------
             2

DELETE 1
 timeline_rows
---------------
             0
```

## 4. SQL QUERIES

Each query answers a control-room question; its heading names the requirement served and the SQL features used. Outputs follow Listing 2; Q5, Q10 and Q15 call the functions of Section 5.1.

### Q1. Active incidents, most severe first *(FR-04; selection, CASE in ORDER BY)*

```
SELECT id, title, severity, status FROM incidents WHERE status IN ('VERIFIED', 'RESPONDING')
ORDER BY CASE severity WHEN 'CRITICAL' THEN 1 WHEN 'HIGH' THEN 2 WHEN 'MODERATE' THEN 3 ELSE 4 END, created_at;
```

```
 id |                    title                     | severity |   status
----+----------------------------------------------+----------+------------
  7 | Palar river overflow near Vellore Fort       | CRITICAL | RESPONDING
 10 | Rising water level at Anaicut check dam      | HIGH     | VERIFIED
 11 | Electrical fire in apartment block           | HIGH     | RESPONDING
 14 | Compound wall collapse after heavy rain      | HIGH     | RESPONDING
 15 | Waterlogging under Katpadi railway underpass | HIGH     | VERIFIED
  9 | Waterlogging in Sathuvachari low-lying streets | MODERATE | VERIFIED
```

### Q2. Reports since 11 October with reporter and zone *(FR-04; inner join, left outer join)*

```
SELECT i.id, u.full_name AS reporter,
       coalesce(z.zone_name, '(unassigned)') AS zone, i.status
FROM incidents i
JOIN users u ON u.id = i.reported_by
LEFT JOIN zones z ON z.id = i.zone_id
WHERE i.created_at >= DATE '2026-10-11'
ORDER BY i.id;
```

```
 id |   reporter    |     zone      |   status
----+---------------+---------------+------------
 10 | Karthik Raman | Anaicut       | VERIFIED
 11 | Kavya Reddy   | Vellore Town  | RESPONDING
 12 | Fathima Begum | (unassigned)  | REPORTED
 13 | Joseph Antony | (unassigned)  | REPORTED
 14 | Gopal Iyer    | Pernambut     | RESPONDING
 15 | Kavya Reddy   | Katpadi       | VERIFIED
```

### Q3. Zones with two or more incidents *(FR-13; GROUP BY, HAVING, aggregate FILTER)*

```
SELECT z.zone_name, count(*) AS incidents,
       count(*) FILTER (WHERE i.status IN ('VERIFIED', 'RESPONDING'))
         AS active
FROM incidents i JOIN zones z ON z.id = i.zone_id
GROUP BY z.zone_name HAVING count(*) >= 2
ORDER BY incidents DESC, z.zone_name;
```

```
  zone_name   | incidents | active
--------------+-----------+--------
 Katpadi      |         3 |      1
 Sathuvachari |         2 |      1
 Vellore Town |         2 |      2
```

### Q4. Units with no dispatch since 6 October *(FR-13; outer join with IS NULL (anti-join))*

```
SELECT r.unit_name, r.unit_type, r.availability_status
FROM response_units r
LEFT JOIN dispatches d
       ON d.unit_id = r.id
      AND d.dispatched_at >= DATE '2026-10-06'
WHERE d.unit_id IS NULL
ORDER BY r.unit_name;
```

```
       unit_name        | unit_type | availability_status
------------------------+-----------+---------------------
 Ambulance KPD-02       | MEDICAL   | AVAILABLE
 Katpadi Fire Unit      | FIRE      | AVAILABLE
 Katpadi Police Patrol  | POLICE    | AVAILABLE
 Relief Unit Gudiyatham | RELIEF    | OFF_DUTY
 Vellore North Police   | POLICE    | AVAILABLE
```

### Q5. Relief items at or below the reorder level *(FR-13; function in WHERE and SELECT)*

```
SELECT resource_name, quantity_available AS available, reorder_level, fn_stock_status(id) AS stock_status
FROM resources WHERE fn_stock_status(id) <> 'ADEQUATE' ORDER BY quantity_available;
```

```
      resource_name        | available | reorder_level | stock_status
---------------------------+-----------+---------------+--------------
 Inflatable rescue boats   |         0 |             2 | OUT_OF_STOCK
 Portable generators       |         2 |             3 | LOW
 Water purification tablets |        80 |           100 | LOW
```

### Q6. Units larger than the average of their type *(FR-06; correlated subquery)*

```
SELECT r.unit_name, r.unit_type, r.personnel_count
FROM response_units r
WHERE r.personnel_count >
      (SELECT avg(x.personnel_count)
       FROM response_units x
       WHERE x.unit_type = r.unit_type)
ORDER BY r.unit_type, r.unit_name;
```

```
       unit_name        | unit_type | personnel_count
------------------------+-----------+-----------------
 Vellore Fire and Rescue | FIRE      |              12
 Ambulance VLR-01        | MEDICAL   |               4
 Vellore North Police    | POLICE    |              10
 Relief Unit Ranipet     | RELIEF    |               7
 SDRF Team Alpha         | RESCUE    |              18
```

### Q7. Coordinators who sent units to CRITICAL incidents *(FR-13; nested IN subqueries)*

```
SELECT u.full_name, u.email
FROM users u
WHERE u.id IN
      (SELECT d.dispatched_by FROM dispatches d
       WHERE d.incident_id IN
             (SELECT id FROM incidents
              WHERE severity = 'CRITICAL'))
ORDER BY u.full_name;
```

```
     full_name      |               email
--------------------+-------------------------------------
 Priya Natarajan    | priya.natarajan@dmers.example.org
 Rajesh Subramanian | rajesh.subramanian@dmers.example.org
```

### Q8. Verified incidents still waiting for a unit *(FR-07; NOT EXISTS)*

```
SELECT i.id, i.title, i.severity, z.zone_name FROM incidents i JOIN zones z ON z.id = i.zone_id
WHERE i.status = 'VERIFIED' AND NOT EXISTS (SELECT 1 FROM dispatches d WHERE d.incident_id = i.id)
ORDER BY i.id;
```

```
 id |                    title                     | severity |  zone_name
----+----------------------------------------------+----------+-------------
  9 | Waterlogging in Sathuvachari low-lying streets | MODERATE | Sathuvachari
 10 | Rising water level at Anaicut check dam      | HIGH     | Anaicut
 15 | Waterlogging under Katpadi railway underpass | HIGH     | Katpadi
```

### Q9. Unit utilisation *(FR-13; join, GROUP BY, aggregate FILTER, interval arithmetic, LIMIT)*

```
SELECT r.unit_name, count(*) AS dispatches,
       count(*) FILTER (WHERE d.dispatch_status = 'COMPLETED') AS completed,
       round(sum(extract(epoch FROM d.cleared_at - d.arrived_at) / 3600)
             FILTER (WHERE d.dispatch_status = 'COMPLETED'), 2) AS hours_on_scene
FROM dispatches d JOIN response_units r ON r.id = d.unit_id
GROUP BY r.unit_name ORDER BY dispatches DESC, hours_on_scene DESC NULLS LAST LIMIT 5;
```

```
       unit_name        | dispatches | completed | hours_on_scene
------------------------+------------+-----------+----------------
 Gudiyatham Fire Unit   |          2 |         1 |           3.48
 SDRF Team Alpha        |          2 |         1 |           1.93
 Vellore Fire and Rescue |          2 |         1 |           1.88
 Ambulance AMB-03       |          2 |         1 |           0.65
 Ambulance VLR-01       |          2 |         0 |
```

### Q10. Average response time per zone *(FR-13; AVG over a stored function)*

```
SELECT z.zone_name, count(*) AS incidents_reached,
       round(avg(fn_response_minutes(i.id)), 1) AS avg_minutes
FROM incidents i JOIN zones z ON z.id = i.zone_id
WHERE fn_response_minutes(i.id) IS NOT NULL
GROUP BY z.zone_name
ORDER BY avg_minutes;
```

```
  zone_name   | incidents_reached | avg_minutes
--------------+-------------------+-------------
 Katpadi      |                 2 |        16.0
 Ambur        |                 1 |        21.0
 Gudiyatham   |                 1 |        21.0
 Vellore Town |                 2 |        31.5
 Sathuvachari |                 1 |        32.0
 Walajapet    |                 1 |        54.0
 Pernambut    |                 1 |        58.0
```

### Q11. Timeline of incident 3 *(FR-12; left outer join, string expressions, to_char)*

```
SELECT t.update_no AS no, to_char(t.logged_at, 'DD-Mon HH24:MI') AS logged_at,
       coalesce(t.previous_status, '-') || ' > ' || t.new_status AS change, u.full_name AS logged_by, t.note
FROM incident_updates t LEFT JOIN users u ON u.id = t.logged_by
WHERE t.incident_id = 3 ORDER BY t.update_no;
```

```
 no |  logged_at   |         change          |    logged_by    |             note
----+--------------+-------------------------+-----------------+------------------------------
  1 | 02-Oct 11:20 | - > REPORTED            | Gopal Iyer      | Incident reported
  2 | 02-Oct 11:26 | REPORTED > VERIFIED     | Priya Natarajan | Status set to VERIFIED
  3 | 02-Oct 11:28 | VERIFIED > RESPONDING   | Priya Natarajan | Status set to RESPONDING
  4 | 02-Oct 12:05 | RESPONDING > RESPONDING | Priya Natarajan | Allocated 20 First-aid kits
  5 | 02-Oct 13:30 | RESPONDING > RESPONDING | Priya Natarajan | Allocated 40 Tarpaulin sheets
  6 | 02-Oct 16:10 | RESPONDING > RESOLVED   | Priya Natarajan | Status set to RESOLVED
  7 | 03-Oct 10:00 | RESOLVED > CLOSED       | Priya Natarajan | Status set to CLOSED
```

### Q12. Everyone accountable for incident 7 *(FR-12; UNION of three joins)*

```
SELECT u.full_name, 'reporter' AS role
FROM incidents i JOIN users u ON u.id = i.reported_by
WHERE i.id = 7
UNION
SELECT u.full_name, 'dispatching coordinator'
FROM dispatches d JOIN users u ON u.id = d.dispatched_by
WHERE d.incident_id = 7
UNION
SELECT u.full_name, 'lead of a dispatched unit'
FROM dispatches d
JOIN response_units r ON r.id = d.unit_id
JOIN users u ON u.id = r.lead_user_id
WHERE d.incident_id = 7
ORDER BY role, full_name;
```

```
     full_name      |           role
--------------------+---------------------------
 Priya Natarajan    | dispatching coordinator
 Rajesh Subramanian | dispatching coordinator
 Arun Prakash       | lead of a dispatched unit
 Lakshmi Narayanan  | lead of a dispatched unit
 Mohammed Irfan     | lead of a dispatched unit
 Anitha Selvam      | reporter
```

### Q13. Busiest unit of each type *(FR-13; derived table, window function ROW_NUMBER)*

```
SELECT unit_type, unit_name, dispatches
FROM (SELECT r.unit_type, r.unit_name,
             count(d.unit_id) AS dispatches,
             row_number() OVER (PARTITION BY r.unit_type
               ORDER BY count(d.unit_id) DESC, r.unit_name) AS rn
      FROM response_units r
      LEFT JOIN dispatches d ON d.unit_id = r.id
      GROUP BY r.unit_type, r.unit_name) ranked
WHERE rn = 1
ORDER BY unit_type;
```

```
 unit_type |       unit_name        | dispatches
-----------+------------------------+------------
 FIRE      | Gudiyatham Fire Unit   |          2
 MEDICAL   | Ambulance AMB-03       |          2
 POLICE    | Katpadi Police Patrol  |          1
 RELIEF    | Relief Unit Gudiyatham |          1
 RESCUE    | SDRF Team Alpha        |          2
```

### Q14. Command board for open incidents *(FR-13; two CTEs, left outer joins)*

```
WITH units AS (SELECT incident_id,
                      count(*) FILTER (WHERE dispatch_status = 'ON_SCENE') AS on_scene,
                      count(*) FILTER (WHERE dispatch_status IN ('ASSIGNED', 'EN_ROUTE')) AS on_the_way
               FROM dispatches GROUP BY incident_id),
     relief AS (SELECT incident_id, count(*) AS relief_items FROM resource_allocations GROUP BY incident_id)
SELECT i.id, i.severity, coalesce(u.on_scene, 0) AS on_scene, coalesce(u.on_the_way, 0) AS on_the_way,
       coalesce(r.relief_items, 0) AS relief_items
FROM incidents i LEFT JOIN units u ON u.incident_id = i.id LEFT JOIN relief r ON r.incident_id = i.id
WHERE i.status IN ('VERIFIED', 'RESPONDING') ORDER BY i.id;
```

```
 id | severity | on_scene | on_the_way | relief_items
----+----------+----------+------------+--------------
  7 | CRITICAL |        3 |          0 |            6
  9 | MODERATE |        0 |          0 |            2
 10 | HIGH     |        0 |          0 |            1
 11 | HIGH     |        1 |          0 |            0
 14 | HIGH     |        1 |          2 |            3
 15 | HIGH     |        0 |          0 |            0
```

**Q15. Zone situation view** *(FR-13; VIEW, scalar subquery, stored function)*

```
CREATE OR REPLACE VIEW v_zone_situation AS
SELECT z.zone_name, z.risk_level,
       fn_active_incidents(z.id) AS active_incidents,
       (SELECT count(*) FROM response_units r
        WHERE r.zone_id = z.id
          AND r.availability_status = 'AVAILABLE')
        AS units_available
FROM zones z;
SELECT * FROM v_zone_situation
WHERE active_incidents > 0 OR risk_level = 'HIGH'
ORDER BY active_incidents DESC, zone_name;
```

```
CREATE VIEW
  zone_name   | risk_level | active_incidents | units_available
--------------+------------+------------------+----------------
 Vellore Town | HIGH       |                2 |               1
 Anaicut      | MEDIUM     |                1 |               0
 Katpadi      | MEDIUM     |                1 |               3
 Pernambut    | MEDIUM     |                1 |               0
 Sathuvachari | HIGH       |                1 |               0
 Gudiyatham   | HIGH       |                0 |               0
```

# 5. PROCEDURES AND FUNCTIONS

## 5.1 Functions

The functions compute the derived attributes that Review I chose not to store (DD-08). They are STABLE: they never modify data and see the snapshot of the calling statement, so Listing 2 reads the stock status in a separate SELECT after the restocking UPDATE.

**Listing 3. Functions F1–F3 (02_plpgsql.sql) and a direct call**

```
-- [F1] fn_stock_status: derived stock_status (BR-10)
CREATE OR REPLACE FUNCTION fn_stock_status(
    p_resource_id INTEGER)
RETURNS VARCHAR LANGUAGE plpgsql STABLE AS $$
DECLARE
  v_qty INTEGER;  v_reorder INTEGER;
BEGIN
  SELECT quantity_available, reorder_level
    INTO v_qty, v_reorder
    FROM resources WHERE id = p_resource_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Resource % does not exist',
                    p_resource_id;
  END IF;
  RETURN CASE WHEN v_qty = 0 THEN 'OUT_OF_STOCK'
              WHEN v_qty <= v_reorder THEN 'LOW'
              ELSE 'ADEQUATE' END;
END $$;
```

```
-- [F2] fn_active_incidents: derived active_incidents
CREATE OR REPLACE FUNCTION fn_active_incidents(
    p_zone_id INTEGER)
RETURNS INTEGER LANGUAGE plpgsql STABLE AS $$
DECLARE
  v_count INTEGER;
BEGIN
  SELECT count(*) INTO v_count FROM incidents
  WHERE zone_id = p_zone_id
    AND status IN ('VERIFIED', 'RESPONDING');
  RETURN v_count;
END $$;
```

```
-- [F3] fn_response_minutes: report to first arrival
CREATE OR REPLACE FUNCTION fn_response_minutes(
    p_incident_id INTEGER)
RETURNS NUMERIC LANGUAGE plpgsql STABLE AS $$
DECLARE
  v_reported TIMESTAMPTZ;  v_arrival TIMESTAMPTZ;
BEGIN
  SELECT created_at INTO v_reported
    FROM incidents WHERE id = p_incident_id;
  SELECT min(arrived_at) INTO v_arrival
    FROM dispatches WHERE incident_id = p_incident_id;
  IF v_reported IS NULL OR v_arrival IS NULL THEN
    RETURN NULL;          -- unknown, or no unit on scene
  END IF;
  RETURN round(extract(epoch FROM v_arrival - v_reported)
               / 60, 1);
END $$;
```

```
SELECT fn_stock_status(11) AS boats, fn_stock_status(12) AS generators,
       fn_active_incidents(1) AS active_in_vellore_town, fn_response_minutes(7) AS palar_response_min;
```

```
    boats     | generators | active_in_vellore_town | palar_response_min
--------------+------------+------------------------+--------------------
 OUT_OF_STOCK | LOW        |                      2 |               41.0
```

## 5.2 Procedure sp_dispatch_unit

*sp_dispatch_unit* carries out FR-07 in one transaction: it checks that the caller is a coordinator, records the dispatch, marks the unit DEPLOYED and moves a VERIFIED incident to RESPONDING. The BR-05 checks sit in trigger T3, so they also cover statements that bypass the procedure.

**Listing 4. Procedure P1 (02_plpgsql.sql), a successful call and three rejected calls**

```
-- [P1] sp_dispatch_unit: dispatch a unit (FR-07)
CREATE OR REPLACE PROCEDURE sp_dispatch_unit(
    p_incident_id INTEGER, p_unit_id INTEGER,
    p_coordinator_id INTEGER)
LANGUAGE plpgsql AS $$
DECLARE
  v_role   users.account_type%TYPE;
  v_status incidents.status%TYPE;
  v_unit   response_units.unit_name%TYPE;
BEGIN
  SELECT account_type INTO v_role
    FROM users WHERE id = p_coordinator_id;
  IF v_role IS DISTINCT FROM 'COORDINATOR' THEN
    RAISE EXCEPTION       -- only coordinators (FR-07)
      'User % is not authorised to dispatch units',
      p_coordinator_id;
  END IF;
  -- author recorded by the timeline trigger T1
  PERFORM set_config('dmers.actor_id',
                     p_coordinator_id::TEXT, true);
```

```
  -- trigger T3 checks BR-05 on this INSERT
  INSERT INTO dispatches (incident_id, unit_id,
                          dispatched_by)
  VALUES (p_incident_id, p_unit_id, p_coordinator_id);
  UPDATE response_units
     SET availability_status = 'DEPLOYED'
   WHERE id = p_unit_id RETURNING unit_name INTO v_unit;

  SELECT status INTO v_status
    FROM incidents WHERE id = p_incident_id;
  IF v_status = 'VERIFIED' THEN       -- first unit sent
    UPDATE incidents SET status = 'RESPONDING'
     WHERE id = p_incident_id;
  END IF;
  RAISE NOTICE 'Unit % (%) dispatched to incident % '
    'by user %', p_unit_id, v_unit, p_incident_id,
    p_coordinator_id;
END $$;
```

```
CALL sp_dispatch_unit(9, 2, 3);
SELECT d.dispatch_status,
       r.availability_status AS unit_now,
       i.status AS incident_now
FROM dispatches d
JOIN response_units r ON r.id = d.unit_id
JOIN incidents i ON i.id = d.incident_id
WHERE d.incident_id = 9 AND d.unit_id = 2;
```

```
NOTICE:  Unit 2 (Katpadi Fire Unit) dispatched to incident 9 by user 3
CALL
 dispatch_status | unit_now | incident_now
-----------------+----------+--------------
 ASSIGNED        | DEPLOYED | RESPONDING
```

```
CALL sp_dispatch_unit(10, 9, 3);     -- SDRF Team Alpha is still deployed at incident 7
CALL sp_dispatch_unit(12, 5, 3);     -- incident 12 has not been verified
CALL sp_dispatch_unit(10, 5, 10);    -- user 10 is a citizen
```

```
ERROR:  Unit 9 is DEPLOYED, not AVAILABLE
ERROR:  Incident 12 is REPORTED; only VERIFIED or RESPONDING incidents receive units
ERROR:  User 10 is not authorised to dispatch units
```

# 6. CURSOR AND TRIGGERS

## 6.1 Cursor in sp_close_incident

The explicit cursor *c_open* selects the incident's active dispatches FOR UPDATE, locking them while the loop runs. Each FETCH returns one dispatch; UPDATE … WHERE CURRENT OF completes it if the unit is on scene or cancels it otherwise, and the unit returns to AVAILABLE. After CLOSE the incident moves to RESOLVED and CLOSED, which T1 records as two timeline entries (FR-11).

**Listing 5. Procedure P2 with its cursor (02_plpgsql.sql) and its execution on incident 14**

```
-- [P2] sp_close_incident: cursor releases units (FR-11)
CREATE OR REPLACE PROCEDURE sp_close_incident(
    p_incident_id INTEGER, p_coordinator_id INTEGER,
    p_note VARCHAR DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE
  c_open CURSOR FOR       -- active dispatches of incident
    SELECT unit_id, dispatch_status FROM dispatches
     WHERE incident_id = p_incident_id
       AND dispatch_status IN ('ASSIGNED', 'EN_ROUTE',
                               'ON_SCENE')
     ORDER BY unit_id
       FOR UPDATE;
  r_disp     RECORD;
  v_status   incidents.status%TYPE;
  v_released INTEGER := 0;
BEGIN
  SELECT status INTO v_status FROM incidents
   WHERE id = p_incident_id FOR UPDATE;
  IF v_status IS NULL OR v_status NOT IN
     ('VERIFIED', 'RESPONDING', 'RESOLVED') THEN
    RAISE EXCEPTION
      'Incident % cannot be closed from status %',
      p_incident_id, coalesce(v_status, 'missing');
  END IF;
  PERFORM set_config('dmers.actor_id',
                     p_coordinator_id::TEXT, true);

  OPEN c_open;
  LOOP
    FETCH c_open INTO r_disp;
    EXIT WHEN NOT FOUND;
    UPDATE dispatches
       SET dispatch_status =
             CASE WHEN r_disp.dispatch_status = 'ON_SCENE'
                  THEN 'COMPLETED' ELSE 'CANCELLED' END,
           cleared_at = now()
     WHERE CURRENT OF c_open;
    UPDATE response_units
       SET availability_status = 'AVAILABLE'
     WHERE id = r_disp.unit_id;
    v_released := v_released + 1;
    RAISE NOTICE 'Unit % released (dispatch was %)',
      r_disp.unit_id, r_disp.dispatch_status;
  END LOOP;
  CLOSE c_open;

  IF v_status <> 'RESOLVED' THEN
    UPDATE incidents SET status = 'RESOLVED'
     WHERE id = p_incident_id;
  END IF;
  -- the closing note is stored with the CLOSED entry
  PERFORM set_config('dmers.note',
                     coalesce(p_note, ''), true);
  UPDATE incidents SET status = 'CLOSED'
   WHERE id = p_incident_id;
  PERFORM set_config('dmers.note', '', true);
  RAISE NOTICE 'Incident % closed; % unit(s) released',
    p_incident_id, v_released;
END $$;
```

```
CALL sp_close_incident(14, 3, 'Debris cleared; structure declared safe');
SELECT d.unit_id, r.unit_name, d.dispatch_status, r.availability_status
FROM dispatches d JOIN response_units r ON r.id = d.unit_id
WHERE d.incident_id = 14 ORDER BY d.unit_id;
SELECT update_no, previous_status, new_status, note, logged_by
FROM incident_updates WHERE incident_id = 14 AND update_no >= 7 ORDER BY update_no;
```

```
NOTICE:  Unit 3 released (dispatch was EN_ROUTE)
NOTICE:  Unit 6 released (dispatch was ASSIGNED)
NOTICE:  Unit 10 released (dispatch was ON_SCENE)
NOTICE:  Incident 14 closed; 3 unit(s) released
CALL
 unit_id |      unit_name       | dispatch_status | availability_status
---------+----------------------+-----------------+---------------------
       3 | Gudiyatham Fire Unit | CANCELLED       | AVAILABLE
       6 | Ambulance AMB-03     | CANCELLED       | AVAILABLE
      10 | SDRF Team Bravo      | COMPLETED       | AVAILABLE

 update_no | previous_status | new_status |                  note                   | logged_by
-----------+-----------------+------------+-----------------------------------------+-----------
         7 | RESPONDING      | RESOLVED   | Status set to RESOLVED                  |         3
         8 | RESOLVED        | CLOSED     | Debris cleared; structure declared safe |         3
```

## 6.2 Triggers

Five triggers enforce the rules in Table 3. T4 is an AFTER trigger on purpose: INSERT … ON CONFLICT DO UPDATE fires BEFORE INSERT triggers even when the row ends up updated, so a BEFORE trigger would take the stock twice, whereas an AFTER trigger fires only for the action performed. T5 admits a delete only at trigger depth above one, i.e. the cascade from *incidents*.

**Table 3. Triggers and the rules they enforce**

| ID | Trigger | Timing and event | Rule enforced |
|---|---|---|---|
| T1 | trg_incidents_timeline | AFTER INSERT OR UPDATE OF status ON incidents | FR-03, BR-08: every status change is written to the timeline |
| T2 | trg_incidents_touch | BEFORE UPDATE ON incidents | *updated_at* is maintained unless the statement sets it |
| T3 | trg_dispatches_guard | BEFORE INSERT ON dispatches | BR-05: only an AVAILABLE unit, only to a VERIFIED or RESPONDING incident |
| T4 | trg_allocations_stock | AFTER INSERT OR UPDATE OF quantity OR DELETE ON resource_allocations | FR-10, BR-07: stock follows the allocation; over-allocation is refused |
| T5 | trg_incident_updates_append_only | BEFORE UPDATE OR DELETE ON incident_updates | BR-08: entries are never edited; deletion only by cascade |

**Listing 6. Trigger functions and CREATE TRIGGER statements T1–T5 (02_plpgsql.sql)**

```
-- [T1] timeline of status changes (FR-03, BR-08)
CREATE OR REPLACE FUNCTION trg_fn_incident_timeline()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
  v_note  TEXT := nullif(current_setting('dmers.note',
                                         true), '');
  v_actor TEXT := nullif(current_setting('dmers.actor_id',
                                         true), '');
BEGIN
  IF TG_OP = 'INSERT' THEN          -- entry 1: the report
    INSERT INTO incident_updates (incident_id, update_no,
      new_status, note, logged_by, logged_at)
    VALUES (NEW.id, 1, NEW.status, 'Incident reported',
            NEW.reported_by, NEW.created_at);
  ELSIF NEW.status IS DISTINCT FROM OLD.status THEN
    INSERT INTO incident_updates (incident_id, update_no,
      previous_status, new_status, note, logged_by,
      logged_at)
    SELECT NEW.id, coalesce(max(update_no), 0) + 1,
      OLD.status, NEW.status,
      coalesce(v_note, 'Status set to ' || NEW.status),
      v_actor::INTEGER, NEW.updated_at
      FROM incident_updates WHERE incident_id = NEW.id;
  END IF;                    -- unchanged status: no entry
  RETURN NULL;              -- ignored in an AFTER trigger
END $$;

CREATE TRIGGER trg_incidents_timeline
AFTER INSERT OR UPDATE OF status ON incidents
FOR EACH ROW EXECUTE FUNCTION trg_fn_incident_timeline();
```

```
-- [T2] maintains incidents.updated_at
CREATE OR REPLACE FUNCTION trg_fn_touch_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.updated_at = OLD.updated_at THEN  -- not supplied
    NEW.updated_at := now();
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER trg_incidents_touch
BEFORE UPDATE ON incidents
FOR EACH ROW EXECUTE FUNCTION trg_fn_touch_updated_at();
```

```
-- [T3] dispatch guard (BR-05)
CREATE OR REPLACE FUNCTION trg_fn_dispatch_guard()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
  v_unit_state response_units.availability_status%TYPE;
  v_incident   incidents.status%TYPE;
BEGIN
  SELECT availability_status INTO v_unit_state
    FROM response_units WHERE id = NEW.unit_id
     FOR UPDATE;                    -- lock the unit row
  IF v_unit_state IS DISTINCT FROM 'AVAILABLE' THEN
    RAISE EXCEPTION 'Unit % is %, not AVAILABLE',
      NEW.unit_id, coalesce(v_unit_state, 'missing');
  END IF;
  SELECT status INTO v_incident
    FROM incidents WHERE id = NEW.incident_id;
  IF v_incident IS NULL
     OR v_incident NOT IN ('VERIFIED', 'RESPONDING') THEN
    RAISE EXCEPTION 'Incident % is %; only VERIFIED or '
      'RESPONDING incidents receive units',
      NEW.incident_id, coalesce(v_incident, 'missing');
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER trg_dispatches_guard
BEFORE INSERT ON dispatches
FOR EACH ROW EXECUTE FUNCTION trg_fn_dispatch_guard();
```

```
-- [T4] stock movement and timeline note (FR-10, BR-07)
CREATE OR REPLACE FUNCTION trg_fn_allocation_stock()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
  v_delta INTEGER;  v_iid INTEGER;  v_rid INTEGER;
  v_left  INTEGER;  v_next INTEGER;
  v_item   resources.resource_name%TYPE;
  v_status incidents.status%TYPE;
BEGIN
  IF TG_OP = 'DELETE' THEN    -- withdrawn: stock returns
    v_delta := -OLD.quantity;
    v_iid := OLD.incident_id;  v_rid := OLD.resource_id;
  ELSE                        -- new allocation or increase
    v_delta := NEW.quantity - CASE TG_OP
      WHEN 'UPDATE' THEN OLD.quantity ELSE 0 END;
    v_iid := NEW.incident_id;  v_rid := NEW.resource_id;
  END IF;
  IF v_delta = 0 THEN RETURN NULL; END IF;
  UPDATE resources
     SET quantity_available = quantity_available - v_delta
   WHERE id = v_rid AND quantity_available >= v_delta
  RETURNING quantity_available, resource_name
       INTO v_left, v_item;
  IF NOT FOUND THEN
    SELECT quantity_available INTO v_left
      FROM resources WHERE id = v_rid;
    RAISE EXCEPTION 'Insufficient stock for resource %: '
      'requested %, available %', v_rid, v_delta, v_left;
  END IF;
  -- lock the incident so timeline numbers stay unique
  SELECT status INTO v_status FROM incidents
   WHERE id = v_iid FOR UPDATE;
  SELECT coalesce(max(update_no), 0) + 1 INTO v_next
    FROM incident_updates WHERE incident_id = v_iid;
  INSERT INTO incident_updates (incident_id, update_no,
    previous_status, new_status, note, logged_by,
    logged_at)
  VALUES (v_iid, v_next, v_status, v_status,
    CASE WHEN v_delta > 0 THEN 'Allocated '
         ELSE 'Returned ' END
      || abs(v_delta) || ' ' || v_item,
    nullif(current_setting('dmers.actor_id', true),
           '')::INTEGER,
    CASE WHEN TG_OP = 'DELETE' THEN now()
         ELSE NEW.allocated_at END);
  RETURN NULL;
END $$;

CREATE TRIGGER trg_allocations_stock
AFTER INSERT OR UPDATE OF quantity OR DELETE
ON resource_allocations
FOR EACH ROW EXECUTE FUNCTION trg_fn_allocation_stock();
```

```
-- [T5] append-only timeline (BR-08)
CREATE OR REPLACE FUNCTION trg_fn_timeline_append_only()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'DELETE' AND pg_trigger_depth() > 1 THEN
    RETURN OLD;      -- cascade from DELETE on incidents
  END IF;
  RAISE EXCEPTION 'Timeline entries are append-only '
    '(incident %, update %)', OLD.incident_id,
    OLD.update_no;
END $$;

CREATE TRIGGER trg_incident_updates_append_only
BEFORE UPDATE OR DELETE ON incident_updates
FOR EACH ROW
EXECUTE FUNCTION trg_fn_timeline_append_only();
```

**Listing 7. Triggers T4 and T5 at work (06_plpgsql_demo.sql)**

```
SET dmers.actor_id = '2';
SELECT quantity_available AS cans_in_stock FROM resources WHERE id = 3;
INSERT INTO resource_allocations AS ra (incident_id, resource_id, quantity) VALUES (7, 3, 40)
ON CONFLICT (incident_id, resource_id)
DO UPDATE SET quantity = ra.quantity + EXCLUDED.quantity, allocated_at = now();
SELECT ra.quantity AS allocated_to_7, r.quantity_available AS cans_in_stock, t.note
FROM resource_allocations ra JOIN resources r ON r.id = ra.resource_id
JOIN incident_updates t ON t.incident_id = ra.incident_id
WHERE ra.incident_id = 7 AND ra.resource_id = 3 ORDER BY t.update_no DESC LIMIT 1;
INSERT INTO resource_allocations (incident_id, resource_id, quantity) VALUES (10, 12, 5);
```

```
SET
 cans_in_stock
---------------
           600

INSERT 0 1
 allocated_to_7 | cans_in_stock |                  note
----------------+---------------+----------------------------------------
            340 |           560 | Allocated 40 Drinking water cans (20 L)

ERROR:  Insufficient stock for resource 12: requested 5, available 2
```

```
UPDATE incident_updates SET note = 'edited'
WHERE incident_id = 3 AND update_no = 1;
DELETE FROM incident_updates WHERE incident_id = 3;
```

```
ERROR:  Timeline entries are append-only (incident 3, update 1)
ERROR:  Timeline entries are append-only (incident 3, update 1)
```

## 6.3 Execution and Transaction Behaviour

**Table 4. Execution flow of the main operations, from statement to result**

| Operation (actor, FR) | Executed | Tables changed and rules applied | Result |
|---|---|---|---|
| Report an incident (citizen, FR-03) | INSERT INTO incidents | *incidents* with DEFAULT status and severity; T1 adds timeline entry 1 | New row (Listing 2) |
| Verify a report (coordinator, FR-05) | UPDATE incidents | A CHECK demands a zone; T2 stamps *updated_at*; T1 logs the change and its author | Updated row (Listing 2) |
| Dispatch a unit (coordinator, FR-07) | CALL sp_dispatch_unit | *dispatches* (guarded by T3 and the partial index), *response_units*, *incidents* (T2, T1) | NOTICE, or an error and no change (Listing 4) |
| Allocate relief (coordinator, FR-10) | INSERT … ON CONFLICT DO UPDATE | *resource_allocations*; T4 takes the stock only if enough remains and adds a timeline note | New total, or an error (Listing 7) |
| Close an incident (coordinator, FR-11) | CALL sp_close_incident | The cursor completes or cancels each active dispatch and frees its unit; T1 logs RESOLVED and CLOSED | NOTICE per unit released (Listing 5) |
| Situation report (coordinator, FR-13) | SELECT with F1–F3, view | None (read only); the STABLE functions run once per row | Report rows (Q5, Q10, Q15) |

Each CALL or DML statement is one transaction in psql's autocommit mode, and the triggers it fires run inside it, so a RAISE EXCEPTION anywhere undoes the whole statement: the rejected calls of Listing 4 and the refused allocation of Listing 7 leave no partial dispatch, stock change or timeline entry. Row locks (SELECT … FOR UPDATE in T3, T4 and the cursor) serialise concurrent dispatches of one unit and the numbering of one incident's timeline; the transaction-local setting *dmers.actor_id* tells the triggers who made each change.

# 7. CONCLUSION AND NEXT STEPS

Review II implements the Review I design in PostgreSQL 16: eight relations with 8 primary keys, 11 foreign keys, 5 unique and 24 check constraints and a partial unique index; data loaded through the triggers; fifteen queries; and two procedures, three functions, one cursor and five triggers, each tied to a requirement or business rule. Review III will connect the Flask backend (psycopg) to this schema, align the registration endpoint's account types with *ck_users_account_type*, and demonstrate at least five transactions from the application.

# REFERENCES

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[2] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation* (Chapter 43, PL/pgSQL; CREATE TRIGGER; INSERT … ON CONFLICT), 2023. https://www.postgresql.org/docs/16/

[3] VIT Vellore, “Course Project Guidelines – Database Systems (BCSE302L),” Fall Semester 2026–27.